위험회피회계 자동화 프로토타입 개발문서

헥사고날 구조와 DDD 관점을 바탕으로 금융 도메인 로직, API, UI, 테스트를 연결한 구현 문서입니다.

STACK

Java 25 · Spring Boot 4 · React 19 · TypeScript

ARCHITECTURE

adapter · application · domain
BigDecimal · Append-only · Tests

01 ARCHITECTURE

시스템 구조

프런트는 사용자 흐름, 백엔드는 회계 유스케이스와 도메인 계산을 담당합니다.

Frontend

React/Vite 기반 UI가 계약 등록, 평가 실행, 관계 지정, 테스트 실행, 분개 조회를 단계별로 제공합니다.

React · Zod · TanStack Query

Backend

Spring Boot API가 공정가치 평가, 헤지 지정, 유효성 테스트, 자동 분개 생성을 담당합니다.

Spring Boot · JPA · PostgreSQL

```
frontend
  -> adapter/web (Controller, DTO)
  -> application (UseCase, Service, Transaction)
  -> domain (Pricing, Policy, Journal Generator)
  -> repository/JPA
  -> PostgreSQL
```

계층	책임	대표 파일
adapter	HTTP 요청/응답, DTO 변환	HedgeDesignationController
application	유스케이스 오케스트레이션	EffectivenessTestService
domain	금융 계산과 회계 분개 규칙	DollarOffsetCalculator, JournalGenerator
repository	JPA 기반 영속성	HedgeRelationshipRepository

02 DOMAIN ENGINE

회계 엔진 구현

금융 계산과 회계 분개는 도메인 로직으로 격리해 테스트 가능하게 만들었습니다.

FX Forward

IRP 기반 이론 선물환율, 할인계수, 공정가치 변동을 계산합니다.

IRS FVH

IRS 평가손익과 고정금리채권 장부조정을 K-IFRS 6.5.8 흐름으로 처리합니다.

Dollar-offset

경제적 관계와 참고 범위를 분리해 PASS/WARNING/FAIL을 판정합니다.

엔진	구현 내용	근거
FX Forward 평가	IRP, 현가계수, 공정가치 계산	K-IFRS 1113 Level 2
CFH Lower of Test	유효분 OCI, 비효과분 P/L 분리	K-IFRS 1109 6.5.11
IRS FVH 분개	수단 평가손익과 피헤지항목 장부조정	K-IFRS 1109 6.5.8
IRS FVH 상각	장부금액 조정 잔액의 후속 상각	K-IFRS 1109 6.5.9

80~125%는 IFRS 9에서 합격선이 아니라 참고 범위로 처리했습니다. FAIL은 경제적 관계가 훼손되는 동방향 변동 등 더 본질적인 조건에 연결했습니다.

API와 화면 흐름

심사자가 직접 따라갈 수 있도록 화면 순서를 업무 순서와 맞췄습니다.

1
평가
/valuations
/api/irs/valuate

2
지정
/hedge-relationships

3
검증
/effectiveness-tests

4
분개
/journal-entries

업무	대표 API	화면
FX Forward 평가	POST /api/v1/valuations/fx-forward	공정가치 평가
IRS 평가	POST /api/irs/valuate	공정가치 평가 - IRS 탭
헤지 지정	POST /api/v1/hedge-relationships	헤지 지정
유효성 테스트	POST /api/v1/effectiveness-tests	유효성 테스트
자동 분개	GET /api/v1/journal-entries	자동 분개
IRS 상각	POST /api/v1/journal-entries/irs-fvh-amortization	자동 분개 - IRS 상각 카드

UI 개선

사이드바 순서와 페이지 플로우를 동일하게 맞추고, 분개 표는 계정명이 한 줄로 보이도록 정리했습니다.

입력 방어

Zod 폼 검증, 백엔드 Bean Validation, 서비스 계층 BusinessException을 함께 사용합니다.

04 AI PIPELINE

AI 에이전트 기반 개발 방식

AI를 결과 생성 도구가 아니라 역할별 검토 파이프라인으로 사용했습니다.

역할	담당	산출물
accounting-expert	K-IFRS 조항, 회계 요구사항, 경계 케이스 검토	요구사항 문서
backend-developer	도메인 로직, API, 테스트 구현	Spring Boot 코드
frontend-developer	사용자 흐름, 화면, 폼 검증 구현	React UI
validator	오류 재현, 빌드/테스트 검증, 누락 탐지	검증 리포트
Codex	로컬 파일 수정, 실행 검증, GitHub 제출 정리	최종 커밋

하이브리드 RAG 접근

전통적인 RAG 파이프라인 전체를 제품 기능으로 만든 것이 아니라, K-IFRS 원문·정리 문서·검색 API를 회계사 에이전트가 교차 참조하도록 구성했습니다. 검색 실패 시에는 문서화된 지식 파일을 보조 근거로 사용하고, 검토 필요 항목은 구현이 아니라 후속 검증 대상으로 분리했습니다.